05_lineage_docs

8. Lineage Strategy

Lineage is not a diagram Helios draws after the fact; it is a property the dbt project *enforces by construction*. Every edge in the DAG exists because one model named another with `ref()` or named a source with `source()`. Because the agents never hand-write SQL — they compose governed metrics that resolve to marts that resolve, transitively, back to raw GA4 columns — that unbroken `ref()` / `source()` chain is literally the artifact that proves the product's headline guarantee: **0 hallucinated columns, 100% governed SQL**. This section specifies how the DAG is built, declared, governed, and traced end-to-end into the semantic layer.

8.1 `ref()` and `source()` build the DAG — never hard-code table names

A relation is referenced exactly two ways, and physical dataset/table names appear in exactly one place each.

```
-- staging/stg_ga4__events.sql : the ONLY place the raw shards are named (via source())
with raw as (
    select * from {{ source('src_ga4', 'events') }}   -- resolves to bigquery-public-
        data.ga4_obfuscated_sample_ecommerce.events_*
    where {{ ga4_shard_filter() }}                     -- prunes _TABLE_SUFFIX shards
)
...

-- intermediate/int_ga4__sessionized.sql : references the staging model, never the raw table
with events as (
    select * from {{ ref('stg_ga4__events') }}
),
params as (
    select * from {{ ref('stg_ga4__event_params') }}
)
...

-- marts/core/fct_funnel.sql : references intermediate, never staging or source
select * from {{ ref('int_ga4__funnel_steps') }}
  join {{ ref('int_ga4__sessionized') }} using (session_key)
```

Rules that keep lineage truthful:

- **`source()` appears only in staging.** `src_ga4.events` is the single declared upstream. No model below staging may `source()`; `dbt_project_evaluator`'s `fct_source_fanout` /

`fct_staging_dependent_on_source` flags any violation in CI.

- **`ref()` everywhere else.** dbt resolves `ref()` to the environment-correct relation (`helios_dev.*` vs `helios_prod.*`) at compile time, so the same SQL is portable across dev/CI/prod. Hard-coding `helios_prod.fct_funnel` would (a) break the DAG edge — dbt wouldn't know to build the parent first — and (b) silently run dev models against prod data. Both are CI failures.
- **One direction only.** `source → staging → intermediate → marts → semantic`. No model `ref()`s a model in its own or a downstream layer. The layered contract is enforced by `dbt_project_evaluator` (`fct_model_directory_get`, `fct_rejoining_of_upstream_concepts`) plus naming-convention checks.

The payoff: the DAG is *derived*, not maintained. `dbt parse` reads the `ref()` / `source()` calls and writes the graph into `manifest.json`. Nobody edits a lineage file.

8.2 Exposures — the 7 agents + the Decision Brief as declared downstream consumers

dbt's DAG normally ends at marts. Helios extends it one hop further with **exposures**, which declare the non-dbt consumers so that "what feeds the Decision Brief?" and "what breaks if I change `fct_funnel`?" have machine-checkable answers. The seven agents and the Decision Brief are each an exposure whose `depends_on` lists the marts (and the semantic models) they consume through `semantic-mcp`.

```yaml
# models/exposures/exposures.yml
version: 2

exposures:
  - name: helios_orchestrator
    label: "Agent: Orchestrator (Opus)"
    type: application
    maturity: high
    url: https://helios.internal/agents/orchestrator
    description: >
      Plan-execute-critique controller for the autonomous run. Reads no marts directly;
      depends on the semantic layer transitively through every worker agent.
    depends_on:
      - ref('fct_daily_funnel')
      - ref('semantic_layer')          # the MetricFlow semantic models node
    owner: {name: Helios AE Team, email: ae@pristineforests.com}
    meta: {agent_model: claude-opus, mcp_allowlist: [semantic, stats, report]}

  - name: helios_monitor
    label: "Agent: Monitor (Sonnet)"
    type: application
    maturity: high
    url: https://helios.internal/agents/monitor
```

```yaml
    description: "Detects funnel anomalies day-over-day; feeds Decompose. Consumes the additive
      daily grain."
    depends_on: [ref('fct_daily_funnel')]
    owner: {name: Helios AE Team, email: ae@pristineforests.com}

  - name: helios_decompose
    label: "Agent: Decompose (Sonnet)"
    type: application
    maturity: high
    url: https://helios.internal/agents/decompose
    description: "Mix vs rate vs interaction decomposition of funnel deltas via stats-mcp."
    depends_on: [ref('fct_daily_funnel'), ref('fct_funnel_by_dim')]
    owner: {name: Helios AE Team, email: ae@pristineforests.com}

  - name: helios_diagnose
    label: "Agent: Diagnose (Opus)"
    type: application
    maturity: high
    url: https://helios.internal/agents/diagnose
    description: "Root-cause hypothesis ranking; slices fct_funnel/fct_sessions by canonical
      dims."
    depends_on: [ref('fct_funnel'), ref('fct_sessions'), ref('fct_cohorts')]
    owner: {name: Helios AE Team, email: ae@pristineforests.com}

  - name: helios_prescribe
    label: "Agent: Prescribe (Sonnet)"
    type: application
    maturity: medium
    url: https://helios.internal/agents/prescribe
    description: "Powered experiment backlog from observed rates/variances."
    depends_on: [ref('fct_funnel'), ref('fct_orders')]
    owner: {name: Helios AE Team, email: ae@pristineforests.com}

  - name: helios_critic
    label: "Agent: Critic (Opus)"
    type: application
    maturity: high
    url: https://helios.internal/agents/critic
    description: "Adversarial gate; re-checks reconciliation and significance before a finding
      ships."
    depends_on: [ref('fct_funnel'), ref('fct_orders'), ref('fct_daily_funnel')]
    owner: {name: Helios AE Team, email: ae@pristineforests.com}

  - name: helios_narrator
    label: "Agent: Narrator (Sonnet)"
    type: application
    maturity: medium
    url: https://helios.internal/agents/narrator
    description: "Renders governed numbers to prose. No run_query; reads only finalized findings."
    depends_on: [ref('fct_daily_funnel')]
    owner: {name: Helios AE Team, email: ae@pristineforests.com}

  - name: helios_decision_brief
    label: "Helios Decision Brief"
```

```
    type: analysis
    maturity: high
    url: https://helios.internal/briefs/latest
    description: >
      The executive deliverable: anomaly → decomposition → diagnosis → dollar-at-risk →
      prescribed backlog. The terminal consumer of the entire DAG.
    depends_on:
      - ref('fct_funnel')
      - ref('fct_daily_funnel')
      - ref('fct_funnel_by_dim')
      - ref('fct_orders')
      - ref('fct_cohorts')
    owner: {name: Helios Product, email: product@pristineforests.com}
```

`maturity` (`high` / `medium` / `low`) signals consumer stability in the docs site; `type` (`application` / `analysis` / `dashboard` / `ml`) categorizes them. These nodes appear in `dbt docs` and are first-class selection targets (8.4).

8.3 Column-level lineage, the docs DAG, and the lineage artifacts

Model-level lineage is free from `ref()`. **Column-level lineage** (which raw GA4 column ultimately feeds `revenue`?) is reconstructed from two artifacts dbt emits on every run:

- **`target/manifest.json`** — the full graph: every node, its `depends_on.nodes`, its compiled SQL, its tests, contracts, exposures, and `columns` with descriptions. This is the lineage source of truth.
- **`target/catalog.json`** — produced by `dbt docs generate`; the *physical* schema (column names, types, row/byte stats) read back from BigQuery's `INFORMATION_SCHEMA`. The doc site overlays catalog onto manifest so a column shows both its declared description and its warehouse-confirmed type.

Column-level lineage (manifest + compiled-SQL parsing) lets a reviewer trace `revenue` (semantic metric) -> `fct_orders.gross_revenue` -> `stg_ga4__events.purchase_revenue_in_usd` -> `src_ga4.events.ecommerce.purchase_revenue_in_usd`. The hosted-dbt and OSS tools (`sqlglot`-based column lineage, `dbt docs` lineage graph) walk exactly this chain. The static docs DAG is generated by:

```
dbt docs generate            # builds manifest.json + catalog.json
dbt docs serve --port 8080   # interactive lineage graph (--select / --exclude, +N/N+ ancestry)
```

In the served graph, clicking `fct_funnel` highlights its full ancestry (`+fct_funnel`) and descendants including the exposures (`fct_funnel+`). The same `+model` / `model+` operators that drive the graph also drive selection (next).

8.4 Impact analysis with state-based selection

Before any change ships, Helios computes its blast radius against the **deferred prod manifest** (the `manifest.json` from the last prod build, the "state").

```
# What did I change, and everything downstream (children), incl. exposures?
dbt ls --select state:modified+ --state ./prod-manifest

# Everything UPstream of a model (its dependencies) — what must build first?
dbt ls --select +fct_funnel

# A model + its full descendant cone (what a change to it can break)
dbt ls --select fct_funnel+

# Does this change reach the Decision Brief?
dbt ls --select state:modified+,exposure:helios_decision_brief --state ./prod-manifest

# Slim CI: build only changed nodes + children, deferring unbuilt parents to prod
dbt build --select state:modified+ --defer --state ./prod-manifest --favor-state
```

`state:modified` compares node fingerprints (compiled SQL, configs, contracts, macro hashes) to the stored state; the `+` operator extends the selection to descendants. `--defer` means unselected parents are read from their *prod* relations rather than rebuilt, so a one-line change to `int_ga4__sessionized` rebuilds only the sessionization keystone and its cone — not the full three-month history — keeping the CI run inside the 5 GiB byte budget. This is the operational definition of "impact analysis": the set returned by `state:modified+` is exactly the set of things that can change behavior, and intersecting it with `exposure:*` tells you which agents/briefs to re-eval.

8.5 Groups, access, and contracts — governance controls on the lineage edges

Lineage tells you *what* connects; groups/access/contracts decide *who may connect and what they may rely on*. These are the mart-protection layer.

Groups + access partition the project into ownership domains and restrict cross-domain `ref()`.

```
# models/marts/__marts.yml
groups:
  - name: core
    owner: {name: Helios AE Team, email: ae@pristineforests.com}
  - name: finance
    owner: {name: Helios Finance, email: finance@pristineforests.com}

models:
  - name: int_ga4__sessionized
    group: core
```

```yaml
    access: private        # keystone: nothing outside `core` may ref() it
  - name: fct_funnel
    group: core
    access: public         # the semantic layer's primary session grain — stable, depended-on
  - name: fct_orders
    group: finance
    access: protected      # ref-able only within the project (not across dbt Mesh projects)
```

`access` has three levels: **private** (referenceable only within the same group — used for the `int_ga4__*` keystones so no mart bypasses them), **protected** (same dbt project only, the default), **public** (any project, including downstream Mesh projects). Marking `int_ga4__sessionized` private structurally forbids a careless `ref('int_ga4__sessionized')` from a finance mart that should have gone through `fct_funnel`; dbt raises a parse-time error. This is governance encoded in the DAG.

Contracts freeze a model's output schema so downstream consumers (and the semantic layer) cannot be silently broken by an upstream column rename or retype.

```yaml
- name: fct_funnel
  config: {contract: {enforced: true}}
  columns:
    - name: session_key
      data_type: string
      constraints: [{type: not_null}, {type: primary_key}]
    - name: reached_purchase
      data_type: boolean
      constraints: [{type: not_null}]
    - name: session_revenue
      data_type: numeric
```

With `contract.enforced: true`, dbt validates the built relation's columns/types against this declaration *before* it replaces the prod table — a `BREAKING CHANGE` error if `session_revenue` were dropped or its type changed. Because `semantic_layer.yaml` binds metrics to these exact columns, the contract is the structural guarantee behind "0 hallucinated columns": the semantic layer can only reference columns the contract promises exist.

8.6 Cross-project lineage via dbt Mesh (Phase 3, multi-tenant / warehouse-agnostic)

The public sample is single-project. The production design (Bible Phase 3) is multi-tenant: a shared **`helios_core`** dbt project owns staging/intermediate/conformed dims, and each tenant (or each warehouse adapter — BigQuery, Snowflake, Databricks) is a **downstream project** that consumes `helios_core`'s `public` models via cross-project `ref()`.

```yaml
# tenant project: dependencies.yml
projects:
  - name: helios_core            # the upstream producer project

# tenant model
select * from {{ ref('helios_core', 'fct_funnel') }}   -- cross-project ref, version-pinned
```

dbt Mesh stitches the two projects' manifests so lineage and impact analysis span the boundary: a `state:modified+` on `helios_core.fct_funnel` reports the *tenant* exposures it breaks. Only `public`-access models are ref-able across the boundary — which is precisely why `int_ga4__*` is `private` and `fct_funnel` is `public`. **Model versions** (`fct_funnel.v1`, `.v2`) let `helios_core` ship a breaking change while tenants migrate on their own schedule. The honest caveat for the static sample: Mesh is designed but not exercised — there is one project and one warehouse today; the access/version annotations are in place so Phase 3 is a configuration change, not a rewrite.

8.7 Lineage extending into the semantic layer — the chain that proves "0 hallucinated columns"

The DAG does not stop at marts. `semantic_layer.yaml` defines MetricFlow semantic models whose `model:` points at a dbt `ref()` and whose measures/dimensions name *columns of that ref*. Because every entity, dimension, and measure binds to a `ref()`d mart column, MetricFlow's parse step fails if a metric references a column the mart (and its contract) does not expose.

```yaml
# semantic_layer.yaml (MetricFlow) — bound 1:1 to the dbt DAG
semantic_models:
  - name: funnel
    model: ref('fct_funnel')              # ← the lineage edge into the semantic layer
    entities: [{name: session, type: primary, expr: session_key}]
    measures:
      - {name: sessions, agg: count_distinct, expr: session_key}
      - {name: purchasing_sessions, agg: sum, expr: "if(reached_purchase, 1, 0)"}
      - {name: revenue, agg: sum, expr: session_revenue}
metrics:
  - name: session_conversion_rate
    type: ratio
    type_params: {numerator: purchasing_sessions, denominator: sessions}
```

The complete provable chain for one metric:

```
session_conversion_rate            (semantic_layer.yaml metric, composed by semantic-mcp)
  └─ measures purchasing_sessions / sessions
       └─ expr over fct_funnel.reached_purchase, fct_funnel.session_key   (contract-enforced columns
            └─ ref('int_ga4__funnel_steps') reached_* + ref('int_ga4__sessionized') session_key
```

```
                └─ ref('stg_ga4__events') / ref('stg_ga4__event_params')
                    └─ source('src_ga4','events')  =  raw GA4 events_* columns
```

Every hop is a `ref()` / `source()` / `expr` binding recorded in `manifest.json` (dbt) and the MetricFlow `semantic_manifest.json`. When an agent asks `semantic-mcp.build_query('session_conversion_rate', dims=['channel_group'])`, the only columns that can appear are ones this chain validates back to raw GA4. An LLM cannot inject a column that is not in the manifest — the metric simply will not compile. That is what "0 hallucinated columns" means operationally: it is enforced by lineage, not promised by a prompt.

---

9. Documentation Strategy

Documentation in Helios has two distinct registers that must never blur: **technical docs** (what a column physically is — owned by dbt) and **business definitions** (what a metric *means* to the business — owned by the semantic layer). Both are version-controlled, both are required, and both are enforced in CI. A model that lacks a description, a test, or a contract does not merge.

9.1 schema.yml descriptions on every model and every column

Every model file is paired with a `schema.yml` carrying a model-level description and a description for *every* column. Descriptions reference reusable **doc blocks** (`{{ doc( ... ) }}`) so a definition lives once and is cited everywhere.

```yaml
# models/marts/core/_core__models.yml
version: 2

models:
  - name: fct_funnel
    description: '{{ doc("fct_funnel") }}'
    group: core
    access: public
    config:
      contract: {enforced: true}
      persist_docs: {relation: true, columns: true}
    meta:
      owner: ae@pristineforests.com
      maturity: high
      contains_pii: false
      sla: "refreshed by 06:00 UTC; freshness N/A on static public sample"
      grain: "one row per session_key"
    columns:
      - name: session_key
        description: '{{ doc("session_key") }}'
        data_type: string
```

```yaml
      constraints: [{type: not_null}, {type: primary_key}]
      tests: [unique, not_null]
    - name: reached_purchase
      description: '{{ doc("reached_flag") }} Stage: purchase (the terminal funnel step).'
      data_type: boolean
      tests: [not_null]
    - name: channel_group
      description: '{{ doc("channel_group") }}'
      tests:
        - relationships: {to: ref('dim_channels'), field: channel_group}
    - name: session_revenue
      description: "Total purchase_revenue_in_usd attributed to the session (USD; 0 for non-
       purchasing sessions)."
      data_type: numeric
```

Note the column descriptions are not prose duplicates — `session_key`, `reached_flag`, and `channel_group` are *shared* doc blocks, so the monotonic-funnel and session-key conventions are defined once and rendered on every model that surfaces them.

9.2 Doc blocks — reusable definitions in `.md` files

Doc blocks live in `.md` files alongside the models and are the single source of *technical* prose. Defining the funnel-monotonicity rule once means it cannot drift between `fct_funnel`, `fct_daily_funnel`, and `fct_funnel_by_dim`.

```
{# models/marts/core/_core__docs.md #}

{% docs session_key %}
Surrogate session identifier: `TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS
        STRING))))`.
The canonical session grain. `sessions = COUNT(DISTINCT session_key)` — never `COUNT(*)`,
never `FARM_FINGERPRINT`. Cookie-scoped (no logged-in `user_id` in the GA4 sample).
{% enddocs %}

{% docs reached_flag %}
A **max-downstream (monotonic)** funnel flag: TRUE if the session reached this stage **or any
later stage**. This guarantees `sessions ≥ reached_view_item ≥ … ≥ reached_purchase`, so
every step-to-step rate is ≤ 1 by construction. The retired `did_*` naming is forbidden.
{% enddocs %}

{% docs channel_group %}
One of exactly **10** GA4 default channel groups (Direct, Organic Search, Paid Search, Display,
Paid Social, Organic Social, Email, Affiliates, Referral, Other). Derived in the single source of
truth `channel_group_case()` macro from session-scoped source/medium (with `has_gclid` paid
detection), falling back to user first-touch `traffic_source` only when session scope is null.
{% enddocs %}

{% docs fct_funnel %}
One row per session (PK `session_key`). Carries the monotonic `reached_*` flags,
        `session_revenue`,
```

```
and wide session dimensions (channel_group, device_category, country, is_new_user). This is the
semantic layer's **primary session grain** — `fct_daily_funnel` aggregates *this* model (so it
inherits revenue), not `int_ga4__funnel_steps`.
{% enddocs %}
```

9.3 persist_docs — pushing descriptions into BigQuery metadata

`persist_docs: {relation: true, columns: true}` (set project-wide and shown per-model above) writes each model's description to the BigQuery **table description** and each column's description to the **column description** at build time. The benefit: an analyst inspecting `helios_prod.fct_funnel` in the BigQuery console — or any catalog tool reading `INFORMATION_SCHEMA.COLUMN_FIELD_PATHS` — sees the same governed definition without opening the repo. Documentation stops being a thing you have to remember to read.

```yaml
# dbt_project.yml — project-wide default
models:
  helios:
    +persist_docs:
      relation: true
      columns: true
```

9.4 dbt docs generate / serve and the static site

```
dbt docs generate            # manifest.json + catalog.json (catalog reads INFORMATION_SCHEMA)
dbt docs serve               # local interactive site: search, lineage graph, column types
```

In CI the generated `target/` (the static `index.html`, `manifest.json`, `catalog.json`) is published to GitHub Pages / GCS so the docs site is always live and matches `main`. The site renders: model + column descriptions (from schema.yml / doc blocks), the lineage DAG (§8.3), tests per model, contracts, and the exposures with their `maturity` / `owner`. `meta` fields surface as a properties table.

9.5 Model-level meta — owner, maturity, contains_pii, sla

Every model carries structured `meta` (shown in 9.1) so governance is queryable, not tribal:

Field	Purpose	Example
`owner`	Who is paged when it breaks	`ae@pristineforests.com`
`maturity`	Stability signal to consumers	`high` (marts) / `medium` (growth rollups)

Field	Purpose	Example
`contains_pii`	Drives access policy + masking review	`false` (GA4 sample is obfuscated; `user_pseudo_id` is a cookie hash)
`sla`	Freshness/availability promise	"06:00 UTC daily; **N/A on static sample**"

The PII flag is honest about the sample: the obfuscated GA4 export has no real identifiers, so `contains_pii: false` is correct *today*, but the field exists so the multi-tenant production export (real `user_id`, geo) can flip it and trigger the masking-policy review without a schema redesign.

9.6 Two registers: semantic layer = business definitions, dbt = technical

The division of labor is strict and is the answer to "where is the *real* definition of `revenue_per_session`?":

- **`semantic_layer.yaml` `business_definition` / `label` fields are canonical for what a metric MEANS.** Example: `revenue_per_session` — "*Total purchase revenue (USD) divided by total sessions over the window; the headline efficiency metric; computed as SUM(revenue)/SUM(sessions) after grouping, never an average of per-segment ratios.*" This is what the Narrator quotes and what the Critic checks a finding against.
- **dbt schema.yml / doc blocks are canonical for what a column IS** — its type, its grain, its derivation, its source column. Technical, not interpretive.

They are linked by lineage (§8.7): the metric's `business_definition` sits atop a measure that binds to a contract-enforced mart column that traces to raw GA4. Business meaning and physical truth are documented in different files but provably the same object.

9.7 README / runbook

The repo root `README.md` is the operational front door: setup (`dbt deps`, profile/WIF auth), the build commands (`dbt build`, layer selection, `--full-refresh` for the small sample), the env vars (`GOOGLE_APPLICATION_CREDENTIALS`, `HELIOS_WH_TOKEN`), and a **runbook** section — how to respond to a failed freshness check (N/A on the static sample; real on the live export), a contract `BREAKING CHANGE`, or a reconciliation failure (`revenue_reconciles` > 0.5% drift). The runbook links each failure mode to the owning `meta.owner` and the relevant singular test (`assert_funnel_monotonicity`, `assert_session_conversion_rate_bounds`).

9.8 Governance — CODEOWNERS, PR review, docs-in-lockstep

- `CODEOWNERS` maps paths to the `group` owners so a change to `models/marts/finance/**` requires Finance review and `semantic_layer.yaml` requires AE review. This mirrors the dbt

`groups` (§8.5) — ownership is the same in Git and in the DAG.

- **PR review** is mandatory; CI must be green (build + tests + the doc/coverage gate below + the eval gate).
- **Docs-in-lockstep rule (CLAUDE.md):** when an artifact is added or a canonical name changes, the same PR updates `DEPENDENCY_MAP.md`, the Bible's Reference Card, and `CLAUDE.md`. A metric rename that doesn't touch `semantic_layer.yaml`'s `business_definition` and the dbt doc block in the same commit is a review reject. Documentation drift is treated as a build break, not a nicety.

9.9 Enforcing doc + test + contract coverage in CI

Coverage is not aspirational; `dbt_project_evaluator` fails the build when a model is undocumented, untested, or uncontracted.

```yaml
# dbt_project.yml — make coverage rules hard failures
vars:
  dbt_project_evaluator:
    documentation_coverage_target: 100      # every model + column described
    test_coverage_target: 100               # every model has ≥1 test
    primary_key_test_coverage_target: 100   # every model's PK is unique + not_null
models:
  dbt_project_evaluator:
    +severity: error                        # findings fail CI, not just warn
```

```
# CI step (runs against the slim, state:modified+ selection)
dbt build --select package:dbt_project_evaluator --resource-type test
# evaluator surfaces: fct_undocumented_models, fct_models_without_tests,
#                     fct_missing_primary_key_tests, fct_undocumented_columns,
#                     fct_models_without_contracts (custom rule), fct_source_fanout
```

The contract requirement is added as a project rule: every `fct_*` / `dim_*` mart must set `contract.enforced: true`, so a new mart cannot reach the semantic layer (and therefore the agents) without a frozen, documented, tested schema. Combined with the eval gate (no accuracy regression, zero hallucination) and the lineage chain of §8.7, the documentation strategy closes the loop: a column an agent can reference is, by CI construction, a column that is described, typed, contracted, tested, and traceable to raw GA4.